AWS Certified DevOps Engineer – Professional（DOP-C02）　タスク 3.2　#12

スケーリングと疎結合

ターゲット追跡・ステップ・スケジュール・予測スケーリング・ウォームプール・ライフサイクルフック・インスタンスあたりのバックログ・混合インスタンス、ECS サービスの自動スケーリング・キャパシティプロバイダー・Karpenter、Lambda の同時実行（予約済み・プロビジョニング）・API Gateway のスロットリングとキャッシュ・RDS Proxy・DynamoDB のキャパシティモード・Aurora Auto Scaling・キャッシュ戦略、SQS（可視性タイムアウト・DLQ・ロングポーリング・FIFO）・部分的なバッチレスポンス・SNS ファンアウトとフィルター・Kinesis のシャード・Step Functions

各ページの下に、根拠にした資料（AWS の DOP-C02 試験ガイド、EC2 Auto Scaling・ECS・EKS・Lambda・API Gateway・DynamoDB・Aurora・RDS・ElastiCache・SQS・SNS・Kinesis・Step Functions の各ガイド）を書いた。『』の中は日本語版ドキュメントの原文どおり

第1章

EC2 とコンテナのスケーリング

負荷の形に合わせてポリシーを選ぶ。

Auto Scaling のポリシーの選び方

負荷の形	ポリシー	ポイント
ある値に保ちたい	ターゲット追跡	CPU 50%・リクエスト数/ターゲット。アラームは自動。まずこれ
超え方で増やす量を変える	ステップ	シンプル+クールダウンは非推奨（追いつかない）
日時が決まっている	スケジュール	月曜 9 時の前に希望キャパシティを上げる
日・週のパターン+起動が遅い	予測	履歴から予測して先に起動。動的と併用
キューの仕事	バックログ/台でターゲット追跡	ApproximateNumberOfMessages ÷ 台数をカスタムメトリクスに

ウォームプール
初期化済みの控え。停止で置けば EC2 料金なし。起動が非常に遅いときだけ

ライフサイクルフック
起動直後・終了直前に待機（既定 1 時間）。ログ退避・導入 → complete-lifecycle-action

出典：ターゲットトラッキング／ステップおよび簡易スケーリング／スケジュール／予測スケーリング／ウォームプール／ライフサイクルフック／Amazon SQS に基づくスケーリングポリシー／スケーリングクールダウン

コンテナと購入オプション

ECS サービスの自動スケーリング
タスク数を増減（Application Auto Scaling：ターゲット追跡・ステップ・予測）

↓ 置き場所が足りない（PENDING）

キャパシティプロバイダーのマネージドスケーリング
EC2 を増減（CapacityProviderReservation でターゲット追跡）

EKS：Karpenter
ちょうどよいサイズのノードを 1 分未満で。HPA は Pod の数だけ

混合インスタンスグループ	役割
ベース	オンデマンドで最低限を確保
増えた分	スポットで安く（中断に強い処理）
複数タイプ	スポットの容量を取りやすく

出典：Amazon ECS サービスを自動的にスケールする／クラスターの自動スケーリング／Karpenter と Cluster Autoscaler／複数のインスタンスタイプと購入オプション

第2章

サーバーレスとデータ層のスケーリング

同時実行・接続・キャパシティ・キャッシュ。

Lambda と API Gateway

設定	何をする
同時実行の上限	リージョン全体で既定 1,000。超えるとスロットリング
予約済み同時実行	関数の分を確保し上限にもする（0 で停止）。追加料金なし
プロビジョニング	初期化済みの環境でコールドスタートなし。料金あり

API Gateway のスロットリング
トークンバケット（レートとバースト）。超えると 429。使用量プラン+API キーで利用者ごとのレートとクォータ

API キャッシュ
ステージで有効化。TTL 既定 300 秒・最大 3600 秒

出典：Lambda 関数のスケーリング／予約済み同時実行数／プロビジョニングされた同時実行数／リクエストをスロットリングする／REST API のキャッシュ設定

データ層

困りごと	答え	ポイント
Lambda → RDS の接続が多すぎる	RDS Proxy	接続プールを再利用。フェイルオーバーも短く
DynamoDB の負荷が読めない	オンデマンドモード	推奨の既定。Auto Scaling は 2 分続けて超えてから
Aurora の読み取りが増減	Aurora Auto Scaling	レプリカの数を増減。リーダーエンドポイント
キャッシュが古い	書き込みスルー+TTL	遅延読み込みは要るときだけ読むが古くなりうる

出典：Amazon RDS Proxy／DynamoDB のスループットキャパシティ／DynamoDB Auto Scaling／Aurora Auto Scaling／キャッシュ戦略

第3章

疎結合とメッセージング

キューとトピックで切り離す。

SQS の設定と Lambda

設定	ポイント
可視性タイムアウト	既定 30 秒・上限 12 時間。処理時間より長く（短いと二重処理）
DLQ	maxReceiveCount を超えたら移す。同じアカウント・リージョン
ロングポーリング	最大 20 秒待つ。空の応答と費用が減る
FIFO	順序・重複なし。メッセージグループごとに並行



1 件失敗で全件が戻る → ReportBatchItemFailures（部分的なバッチレスポンス）。

出典：可視性タイムアウト／デッドレターキュー／ショートポーリングとロングポーリング／FIFO キュー／Amazon SQS での Lambda の使用

SNS・Kinesis・Step Functions

注文サービス → SNS トピック

↓ ファンアウト（フィルターポリシーで絞る）

在庫キュー ｜ 配送キュー ｜ メールキュー
各自のペースで処理。止まってもキューに残る

> Step Functions：順番・再試行・分岐・戻しを定義で。Lambda どうしの直呼びをほどく。

Kinesis のシャード	上限
書き込み	1 MB/秒・1,000 レコード/秒
読み取り	2 MB/秒・5 トランザクション/秒
パーティションキー	偏るとホットシャード → ばらつくキーに
オンデマンド	シャードを自動で管理

出典：SNS 通知の SQS キューへのファンアウト／メッセージフィルター処理／Kinesis Data Streams の用語と概念／Step Functions とは

本番での問われ方

ねらい	見るところ
EC2	ターゲット追跡・ステップ・スケジュール・予測の選び分け、ウォームプール、ライフサイクルフック、バックログ/台、スポット
コンテナ	タスク数（サービス）と置き場所（キャパシティプロバイダー）、Karpenter
サーバーレス	予約済みとプロビジョニングの違い、429、使用量プラン、API キャッシュ、RDS Proxy
データ	オンデマンドモード、Aurora Auto Scaling、書き込みスルー+TTL
疎結合	可視性タイムアウト、DLQ、ロングポーリング、FIFO とグループ、ファンアウトとフィルター、部分バッチ、シャード、Step Functions

出典：AWS Certified DevOps Engineer - Professional (DOP-C02) 試験ガイド タスク 3.2